R2-A2-S9 Diagrama de Flujo

Grupo #2

Jair Santiago Velasquez Galindo

Gael Antonio Toro Cardozo

Santiago Angel Rodriguez

Jeshua Garcia Guzman

Estructura de datos: G401

Ing. Dilia Inés Molina

Universidad de Cundinamarca

Facultad de ingeniería. Ingeniería de Software

7 de octubre de 2026

INVESTIGACIÓN

TDA (Tipo De Dato Abstracto)

Es un modelo matemático y conceptual que define un conjunto de datos y operaciones permitidas sobre ellos sin revelar su implementación interna, gracias a esto se puede encapsular información y funcionalidades específicas para un propósito determinado.

Características:

- Abstracción: Permite trabajar con los datos sin preocuparse por cómo están implementados internamente.
- Encapsulamiento: Agrupa los datos y operaciones asociadas en una sola unidad lógica.
- Ocultamiento de información: Restringe el acceso directo a la estructura interna de datos, obligando a interactuar únicamente mediante una interfaz.
- Independencia e implementación: Permite cambiar la estructura interna de datos sin alterar el código externo que utiliza el TDA.

Tipos de datos (TDA):

Pila: Funciona como una pila de datos, el ultimo elemento en entrar es el primero en salir. Sus operaciones clave son apilar y desapilar.

Cola: Funciona como una fila de personas. El primer elemento en entrar es el primero en salir, sus operaciones principales son en colar y desencolar.

Lista: Secuencia ordenada de elementos donde se puede insertar, buscar o borrar datos en posiciones específicas.

Diccionario: Almacena información en parejas de clave-valor, permitiendo buscar o insertar valores rápidamente usando una llave.

Arból y grafo: Estructuras no lineales usadas para representar jerarquías o redes de datos conectados entre sí.

Ejemplo pila en diferentes lenguajes:

C

```c
#include <stdio.h>
#define MAX 5

typedef struct {
   int datos[MAX];
   int tope;
} Pila;

void inicializar(Pila *p) {
   p->tope = -1;
}
void push(Pila *p, int valor) {
   if (p->tope < MAX - 1)
       p->datos[++p->tope] = valor;
}
int pop(Pila *p) {
   if (p->tope >= 0)
       return p->datos[p->tope--];
   return -1;
}
int peek(Pila *p) {
   if (p->tope >= 0)
       return p->datos[p->tope];
   return -1;
}

int main() {
   Pila p;
   inicializar(&p);
   push(&p, 10);
   push(&p, 20);
   push(&p, 30);
   printf("Elemento superior: %d\n", peek(&p));
   printf("Eliminado: %d\n", pop(&p));
```

```
    return 0;
}
```

Python

```
class Pila:
    def __init__(self):
        self.datos = []

    def push(self, valor):
        self.datos.append(valor)

    def pop(self):
        if not self.is_empty():
            return self.datos.pop()

    def peek(self):
        if not self.is_empty():
            return self.datos[-1]

    def is_empty(self):
        return len(self.datos) == 0


p = Pila()

p.push(10)
p.push(20)
p.push(30)

print("Superior:", p.peek())
print("Eliminado:", p.pop())
```

Java

```
class Pila {
    private int[] datos = new int[5];
    private int tope = -1;

    public void push(int valor) {
        if (tope < 4) {
            datos[++tope] = valor;
        }
```

```
    }

    public int pop() {
        if (tope >= 0) {
            return datos[tope--];
        }

        return -1;
    }

    public int peek() {
        if (tope >= 0) {
            return datos[tope];
        }

        return -1;
    }
}

public class Main {
    public static void main(String[] args) {
        Pila p = new Pila();

        p.push(10);
        p.push(20);
        p.push(30);

        System.out.println("Superior: " + p.peek());
        System.out.println("Eliminado: " + p.pop());
    }
}
```

Comparación

Lenguaje	Forma de implementar el TDA
C	Struct + funciones
Python	Class + métodos
Java	Class + metodos

Estructuras Lineales Dobles

Es una colección lineal de datos donde cada elemento se almacena en un nodo independiente.

Cada estructura viene en conjuntos de nodos enlazados, cada nodo contiene tres campos, dos llamados enlaces, que son las referencias al nodo siguiente y al anterior en la secuencia de nodos y otro para almacenar información.
El enlace al nodo anterior del primer nodo y enlace de nodo siguiente del último nodo, apuntan a tipo de nodo que marca el final de la lista, nodo centinela o puntero null, esto facilita el recorrido de la lista.

Característica de Estructura lineales dobles:

- Facilidad de inserción y borrado: Las listas comunes requieren desplazar elementos al insertar o borrar (salvo al final), lo que tiene una complejidad de O(n) y afecta el rendimiento en tamaños grandes.

  En cambio, las listas enlazadas almacenan elementos en memoria no contigua y los conectan mediante punteros, lo que permite añadir o eliminar modificando solo los enlaces. Esta operación física es de complejidad O (1) si ya se tiene la referencia directa al nodo (como en la cabeza), aunque encontrar una posición específica requiere un recorrido previo de O(n).
- Tamaño dinámico:

  Las listas de Python son arrays dinámicas que cambian de tamaño reasignando la estructura del bloque de memoria más grande y copiando sus elementos, proceso ineficiente. Por el contrario, las listas enlazadas dobles crecen y se reducen de forma dinámica, fluida sin necesidad de cambiar todo de nuevo en la memoria, lo que requiere más flexibilidad en el manejo de datos.

- Eficiencia de memoria:

  Las listas comunes reservan memoria en un bloque contiguo, lo que resulta ineficiente si crecen (ya que deben copiar todo a un bloque mayor) o si se sobreestima su tamaño (desperdiciando espacio). Por el contrario, las listas enlazadas asignan memoria de forma individual por cada elemento conforme se añaden, optimizando el uso del espacio sin necesidad de redimensionar.

  Otras características:

  - Cada nodo contiene dos punteros, uno al nodo anterior y otro al nodo siguiente. Es posible avanzar y retroceder en la lista enlazada.
  - El puntero anterior del primer nodo apunta a nulo, y el puntero siguiente del último nodo también apunta a nulo.
  - Al principio y al final de la lista, se pueden realizar inserciones y eliminaciones de forma continua.
  - La complejidad de las operaciones de acceso y búsqueda es O(n), donde n es el número de elementos en la lista.
  - Las listas doblemente enlazadas se pueden visitar en ambos sentidos, lo que las hace más flexibles que las listas simplemente enlazadas.

  Tipos de estructura lineal dobles

  - Lista Doblemente Enlazada Lineal (Estándar)

    Es el tipo básico y el más utilizado para aprender la estructura. Los datos se organizan en una línea recta con un principio y un fin claramente definidos.

  - Lista Doblemente Enlazada Circular

En este tipo, la estructura elimina los extremos "sueltos" y conecta el principio con el fin, creando un bucle infinito que se puede recorrer en ambos sentidos.

- Cola de Doble Extremo o Bicola (Deque - Double-Ended Queue)

Es una variante de la estructura lineal doble optimizada específicamente para funcionar como una restricción de flujo. En lugar de enfocarse en recorrer los nodos del medio, está diseñada para operar masivamente en los bordes.

- Lista Doblemente Enlazada con Nodo Centinela(Cabecera/Cola Ficticia)

Es una variante de diseño de software muy utilizada por desarrolladores experimentados para evitar errores de código.

Aspecto	Tipo de Dato Abstracto (TDA)	Estructuras lineales dobles
Definición	Modelo lógico que define los datos y las operaciones permitidas, sin indicar cómo se implementan.	Estructura donde cada nodo guarda enlaces al nodo anterior y al siguiente.
Nivel	Conceptual (qué hace).	De implementación (cómo lo hace).
Ejemplos	Pila, cola, lista, árbol, grafo.	Lista doblemente enlazada, lista doble circular, deque (cola de doble extremo).
Componentes	Datos + operaciones + reglas de uso.	Nodos con dato, puntero anterior y puntero siguiente.
Recorrido	Depende de la implementación elegida.	Bidireccional: de inicio a fin y de fin a inicio.
Operaciones típicas	Insertar, eliminar, buscar, consultar.	Insertar/eliminar al inicio, al final o en medio; recorrer en ambos sentidos.
Ventajas	Ocultamiento de información, reutilización y modularidad.	Eliminación más simple (acceso directo al nodo previo) y recorrido en ambos sentidos.
Desventajas	Necesita una implementación concreta para poder usarse.	Mayor uso de memoria (dos punteros por nodo) y manejo más complejo de enlaces.
Relación	Puede implementarse con estructuras lineales dobles.	Es una forma de implementar TDAs como lista, pila, cola o deque.

ENUNCIADO

Instrucciones

1. Con base en la actividad anterior y teniendo en cuenta los TDA realizar una aplicación usando estructuras lineales dobles
2. Se debe evidenciar el proceso de creación desde la programación orientada a objetos
3. Crear un programa usando el lenguaje de programación JAVA aplicando conceptos de listas dobles.
4. Crear un menú de opciones con a siguiente estructura:

- Insertar cliente: Esta opción permite pedir la cedula y nombre del cliente a ser insertado sobre la lista simple. Esta inserción se deberá realizar de forma ordenada
- Listar Clientes hacia la derecha: Esta opción permitirá imprimir los n clientes que han sido cargados sobre la lista doble desde el primer nodo hasta el ultimo nodo
- Listar Clientes hacia la izquierda: Esta opción permitirá imprimir los n clientes que han sido cargados sobre la lista doble desde el ultimo nodo hasta el primer nodo
- Salir: Esta opción permite finalizar la aplicación

CÓDIGO (Con lenguaje de programación Python)

```python

# CLASE CLIENTE

class Cliente:
    def __init__(self, nombre, cedula):
        self.nombre = nombre
        self.cedula = cedula

    def __str__(self):
        return f"Cédula: {self.cedula} | Nombre: {self.nombre}"

# CLASE NODO DOBLE
class Nodo:
    def __init__(self, cliente):
        self.cliente = cliente
        self.siguiente = None
        self.anterior = None

# CLASE LISTA DOBLE CLIENTES

class ListaClientes:

    def __init__(self):
        self.primero = None
        self.ultimo = None

    # MÉTODO INSERTAR ORDENADO
    def insertar_ordenado(self, cliente):

        nuevo = Nodo(cliente)
```

```
        # CASO 1: la lista está vacía
        if self.primero is None:
            self.primero = nuevo
            self.ultimo = nuevo
            print("\nCliente insertado correctamente.")
            return

        # CASO 2: el cliente debe quedar de primero
        if cliente.cedula < self.primero.cliente.cedula:
            nuevo.siguiente = self.primero
            self.primero.anterior = nuevo
            self.primero = nuevo
            print("\nCliente insertado correctamente.")
            return

        # CASO 3: buscar la posición correcta (en medio o al final)
        actual = self.primero

        while (
            actual.siguiente is not None
            and actual.siguiente.cliente.cedula < cliente.cedula
        ):
            actual = actual.siguiente

        # Insertamos el nuevo nodo reajustando los 4 enlaces
        nuevo.siguiente = actual.siguiente
        nuevo.anterior = actual

        if actual.siguiente is not None:
            # Si va en el medio, enlazamos el anterior del siguiente nodo
            actual.siguiente.anterior = nuevo
        else:
            # Si va al final, el nuevo nodo se convierte en el último
            self.ultimo = nuevo

        actual.siguiente = nuevo

        print("\nCliente insertado correctamente.")

    # MÉTODO LISTAR HACIA LA DERECHA

    def listar_derecha(self):
        if self.primero is None:
            print("\nLa lista de clientes está vacía.")
            return
```

```
        actual = self.primero
        print("\n========================= CLIENTES (HACIA LA DERECHA)
=========================")

        while actual is not None:
            print(actual.cliente, end = " -> ")
            actual = actual.siguiente


print("\n================================================================================
==========")

    # MÉTODO LISTAR HACIA LA IZQUIERDA
    def listar_izquierda(self):

        if self.ultimo is None:
            print("\nLa lista de clientes está vacía.")
            return

        actual = self.ultimo
        print("\n========================= CLIENTES (HACIA LA IZQUIERDA)
=========================")

        while actual is not None:
            print(actual.cliente, end = " -> ")
            actual = actual.anterior


print("\n================================================================================
==========")

    # MÉTODO MOSTRAR ESTRUCTURA DOBLE

    def mostrar_estructura(self):

        if self.primero is None:
            print("\nLa lista está vacía.")
            return

        actual = self.primero
        print("\nEstructura de la lista doble:")
        print("None <- ", end="")

        while actual is not None:
```

```
            print(f"[{actual.cliente.cedula}: {actual.cliente.nombre}]",
end="")

            if actual.siguiente is not None:
                print(" <-> ", end="")
            else:
                print(" -> None")

            actual = actual.siguiente

# FUNCIÓN MENÚ

def mostrar_menu():

    print("\n")
    print("========================================")
    print("       SISTEMA DE CLIENTES")
    print("       LISTA ENLAZADA DOBLE")
    print("========================================")
    print("1. Insertar cliente")
    print("2. Listar clientes hacia la derecha")
    print("3. Listar clientes hacia la izquierda")
    print("4. Salir")
    print("========================================")

# PROGRAMA PRINCIPAL

def main():

    lista = ListaClientes()
    opcion = 0

    while opcion != 4:

        mostrar_menu()

        try:
            opcion = int(input("Seleccione una opción: "))

        except ValueError:
            print("\nDebe ingresar un número.")
            continue

        # OPCIÓN 1: INSERTAR CLIENTE

        if opcion == 1:
```

```
            print("\n========== INSERTAR CLIENTE ==========")
            nombre = input("Ingrese el nombre del cliente: ")

            try:
                cedula = int(input("Ingrese la cédula del cliente: "))
            except ValueError:
                print("\nLa cédula debe ser un número.")
                continue

            cliente = Cliente(nombre, cedula)
            lista.insertar_ordenado(cliente)

        # OPCIÓN 2: LISTAR HACIA LA DERECHA

        elif opcion == 2:
            lista.listar_derecha()
            lista.mostrar_estructura()

        # OPCIÓN 3: LISTAR HACIA LA IZQUIERDA

        elif opcion == 3:
            lista.listar_izquierda()

        # OPCIÓN 4: SALIR

        elif opcion == 4:
            print("\nPrograma finalizado.")
            print("Gracias por utilizar el sistema.")

        # OPCIÓN NO VÁLIDA
        else:
            print("\nOpción no válida.")
            print("Seleccione una opción de la lista (1 al 4).")


# EJECUTAR EL PROGRAMA

if __name__ == "__main__":
    main()
```

SALIDA POR CONSOLA

```
========================================
       SISTEMA DE CLIENTES
       LISTA ENLAZADA DOBLE
========================================
1. Insertar cliente
2. Listar clientes hacia la derecha
3. Listar clientes hacia la izquierda
4. Salir
========================================
Seleccione una opción: 2

La lista de clientes está vacía.

La lista está vacía.
```

```
========================================
       SISTEMA DE CLIENTES
       LISTA ENLAZADA DOBLE
========================================
1. Insertar cliente
2. Listar clientes hacia la derecha
3. Listar clientes hacia la izquierda
4. Salir
========================================
Seleccione una opción: 1

========== INSERTAR CLIENTE ==========
Ingrese el nombre del cliente: Nana
Ingrese la cédula del cliente: 12345

Cliente insertado correctamente.
```

```
========================================
       SISTEMA DE CLIENTES
       LISTA ENLAZADA DOBLE
========================================
1. Insertar cliente
2. Listar clientes hacia la derecha
3. Listar clientes hacia la izquierda
4. Salir
========================================
Seleccione una opción: 1

========== INSERTAR CLIENTE ==========
Ingrese el nombre del cliente: Jeshua
Ingrese la cédula del cliente: 67890

Cliente insertado correctamente.
```

```
========================================
       SISTEMA DE CLIENTES
       LISTA ENLAZADA DOBLE
========================================
1. Insertar cliente
2. Listar clientes hacia la derecha
3. Listar clientes hacia la izquierda
4. Salir
========================================
Seleccione una opción: 1

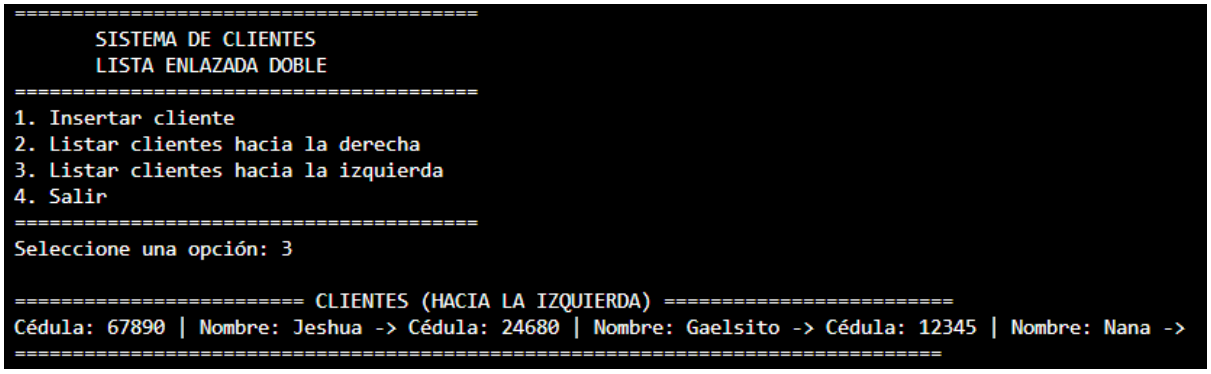
========== INSERTAR CLIENTE ==========
Ingrese el nombre del cliente: Gaelsito
Ingrese la cédula del cliente: 24680

Cliente insertado correctamente.
```

```
========================================
       SISTEMA DE CLIENTES
       LISTA ENLAZADA DOBLE
========================================
1. Insertar cliente
2. Listar clientes hacia la derecha
3. Listar clientes hacia la izquierda
4. Salir
========================================
Seleccione una opción: 2

========================= CLIENTES (HACIA LA DERECHA) =========================
Cédula: 12345 | Nombre: Nana -> Cédula: 24680 | Nombre: Gaelsito -> Cédula: 67890 | Nombre: Jeshua ->
===============================================================================

Estructura de la lista doble:
None <- [12345: Nana] <-> [24680: Gaelsito] <-> [67890: Jeshua] -> None
```



CONCLUSIÓN

Los TDA permiten definir qué operaciones tiene una estructura sin depender de cómo se implementa, y las estructuras lineales dobles son una forma práctica de implementarlos. En el sistema de clientes se vio que la lista doblemente enlazada permite insertar de forma ordenada y recorrer los datos en ambos sentidos.

A cambio, gasta más memoria y exige cuidar los enlaces. Es una buena opción cuando se necesita recorrer en ambas direcciones y modificar datos con frecuencia, a continuación, las ventajas y desventajas de lo anteriormente mencionad**o**.

Ventajas

- Se recorre en ambos sentidos (derecha e izquierda), como en tu programa.
- Insertar y eliminar solo cambia enlaces, sin desplazar elementos (O(1) si ya tienes el nodo).
- El tamaño es dinámico: crece o se reduce sin copiar todo a un bloque nuevo de memoria.
- Eliminar un nodo es más simple, porque cada nodo conoce a su anterior.

Desventajas

- Usa más memoria: dos punteros por nodo.
- Buscar o acceder a una posición es O(n), porque no hay acceso directo por índice.
- Manejar los enlaces es más complejo (en tu inserción ordenada hay que reajustar 4 enlaces) y es fácil cometer errores.

BIBLIOGRAFIA

UAPA. (s.f). *Tipo de Dato Abstracto.* Unidad de Apoyo para el Aprendizaje. UAPA. Tipo de Dato Abstracto

SCRIBD. (s.f). Resumen de tipos de datos abstractos. Tipos de Datos Abstractos en Programación.Tipos de Datos Abstractos en Programación | PDF | Lenguaje de programación | Tipo de datos

Pozo Ramos, L. (2026, 25 de abril). *Estructuras de datos en Python: Listas doblemente enlazadas*. Python Scouts. https://pythonscouts.com/python-doubly-linked-list/

Selvaraj, N. (2026, 22 de julio). *Listas enlazadas en Python: tutorial con ejemplos*. DataCamp. https://www.datacamp.com/es/tutorial/python-linked-lists

Wikipedia. (2026, 25 de septiembre). **Lista doblemente enlazada.** En Wikipedia, la enciclopedia libre. https://es.wikipedia.org/wiki/Lista_doblemente_enlazada